USTED Student Portal v2: Requirements, SRS, UML and Design Documentation

Requirements, final SRS (IEEE 830 style), UML diagrams and design documentation

Derived from the delivered source code, v2.0.0

Contents

Contents 2

Part A: Requirements and SRS 4

1. Introduction . . . 4
2. Overall description . . . 4
3. Stakeholder needs (requirements elicitation) . . . 5
4. Functional requirements . . . 6
5. External interface requirements . . . 8
6. Non-functional requirements . . . 8
7. Business rules . . . 8
8. Data requirements . . . 9
9. Constraints, risks and known limitations . . . 9

Part B: UML diagrams 10

B1. Use case diagram . . . 11
B2. Class diagram (domain model) . . . 12
B3. Entity-relationship diagram (database) . . . 14
B4. Sequence diagram: login with lockout . . . 15
B5. Sequence diagram: course registration . . . 16
B6. Sequence diagram: request submitted and reviewed . . . 17
B7. Sequence diagram: forgotten password . . . 19
B8. Activity diagram: registration eligibility and save . . . 20
B9. State diagrams . . . 21
B10. Component diagram . . . 23
B11. Deployment diagram . . . 25

Part C: Design documentation 27

C1. Design goals . . . 27
C2. Architecture . . . 27
C3. Module design . . . 27
C4. Database design . . . 28
C5. Screens and access control . . . 29
C6. Security design . . . 29
C7. Key algorithms and rules . . . 29
C8. User interface design . . . 29
C9. Validation and error handling . . . 30
C10. Test design . . . 30
C11. Build, run and deployment . . . 30
C12. Requirements traceability . . . 31
C13. Recommended next steps . . . 31

Part A: Requirements and SRS

1. Introduction

1.1 Purpose

This SRS describes the functional and non-functional requirements of the USTED Student Portal v2, a mobile-first portal for students, staff and administrators of the University of Skills Training and Entrepreneurial Development (USTED), Kumasi Campus, Tanoso. It is the baseline for development, testing and acceptance.

1.2 Scope

Students log in with their index number, register for courses, view results, pay fees, submit requests (resits, incomplete grades, complaints, medicals), keep an internship logbook, evaluate courses and read timetables and announcements. Staff and administrators manage students, review requests, mark attendance, publish announcements and export reports. The build runs on Android, iOS (Expo Go) and the web from one JavaScript code base, with all data in an on-device SQLite database created and seeded on first launch.

Out of scope for this release: a central server or API, real payment gateways, real SMS, e-mail or Telegram delivery, and integration with the original PHP and MySQL ERP.

1.3 Definitions

Term	Meaning
SRS	Software Requirements Specification
RBAC	Role-based access control (student, staff, admin)
Index number	A student's unique login name, for example 5211041001
GPA / CGPA	Semester / cumulative grade point average on a 4.0 scale
IC	Incomplete grade request
Resit	Re-sitting a failed course (grade point below 1.5)
SRC levy	Student Representative Council fee added to each invoice (GHS 100)
Term	An academic year and semester, for example 2026/2027-1
Expo / Expo Go	The React Native toolchain and the phone app used to run the portal

1.4 References

- Portal source code v2.0.0 (App.js, src/config.js, src/db, src/screens, src/navigation)
- Portal README (demo accounts, run instructions, production notes)
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications

2. Overall description

2.1 Product perspective

The portal is a self-contained, offline-first client application and an upgrade of the earlier PHP and MySQL app. All data access sits in one file (src/db/repo.js), so a future server API can replace it without changing the screens.

2.2 Product functions

1. Authentication, session restore, lockout, password change and reset
2. Student dashboard, profile and settings
3. Course registration with eligibility checks
4. Results, GPA, CGPA and degree classification
5. Fees, invoices, payments and receipts
6. Student requests: resits, incomplete grades, complaints, medicals, internship
7. Course evaluation, timetable and attendance
8. Notifications and announcements
9. Administration: students, requests, attendance, announcements, reports, registration control, audit trail
10. PDF documents (transcript, receipt, registration slip) and CSV export

2.3 User classes

User class	Login name	Capabilities
Student	Index number (demo: 5211041001 to 5211041010)	All student screens, own data only
Staff	Username (demo: staff)	Dashboard, requests, attendance, announcements, reports, settings, notifications
Administrator	Username (demo: admin)	Everything staff can do, plus student management and reset of demo data

2.4 Operating environment

- Android and iOS through Expo Go, and modern web browsers (permanent sidebar at 900 px width or more, drawer menu below that)
- Expo SDK 58, React Native 0.88, React 19, expo-sqlite, expo-crypto, expo-print, expo-sharing, react-native-web
- Node.js and npm for development

2.5 Design and implementation constraints

- All data is stored locally on each device (SQLite file usted_portal.db)
- Business rules live in one file (src/config.js) and must stay configurable there
- The UI follows the USTED portal look (maroon, teal and blue) and supports light and dark mode

2.6 Assumptions and dependencies

- Users have Expo Go or a browser and open the app once to create the database
- Institution data (programmes, fees, halls, grade scale) is correct as configured in src/config.js
- A production release adds a server, which is a dependency for real security (see section 9)

3. Stakeholder needs (requirements elicitation)

Stakeholder	Need	Resulting requirements
Student	Check results and fees quickly on a phone; register without queues; send requests and see answers	FR-REG, FR-RES, FR-FEE, FR-REQ, FR-NOT
Lecturer / staff	Mark attendance; answer requests; publish notices	FR-ATT, FR-ADM-REQ, FR-ADM-ANN
Registry / administrator	Manage student records and holds; control registration; export data; trace actions	FR-ADM-STU, FR-ADM-SET, FR-ADM-REP, FR-AUD
Finance office	See who has paid; enforce the 60 percent rule; issue receipts	FR-FEE, BR-01, FR-ADM-REP
ICT unit	One code base; configurable rules; a path to a central server	NFR-MNT, NFR-POR

4. Functional requirements

Priority: M = must have, S = should have. Each requirement is implemented in v2.0.0.

4.1 Authentication and account (FR-AUTH)

ID	Requirement	Pri
FR-AUTH-01	The system shall let a user log in with index number (or username) and password; the username is not case-sensitive	M
FR-AUTH-02	The system shall reject deactivated accounts with a message to contact the registry	M
FR-AUTH-03	The system shall lock an account for 10 minutes after 5 consecutive failed logins and show the attempts left	M
FR-AUTH-04	The system shall store passwords only as salted SHA-256 hashes	M
FR-AUTH-05	The system shall restore the last session when the app reopens and let the user log out	M
FR-AUTH-06	The system shall let a logged-in user change their password after confirming the current one	M
FR-AUTH-07	The system shall require passwords of at least 8 characters with a letter and a number, and show a strength meter	M
FR-AUTH-08	The system shall offer forgot-password by Email, SMS or Telegram, issue a 6-digit code valid for 10 minutes, and accept it once	M
FR-AUTH-09	The system shall open the screen set for the user's role (student Home or admin Dashboard) and block screens the role may not open	M

4.2 Student home and profile (FR-PRO)

ID	Requirement	Pri
FR-PRO-01	The Home screen shall show CGPA, fee balance, registered credits, today's classes and latest announcements	M
FR-PRO-02	The Profile screen shall have Personal, Programme and Security tabs	M
FR-PRO-03	A student shall be able to edit only their contact details: e-mail, phone, address, guardian name and guardian phone	M
FR-PRO-04	The Settings screen shall offer light and dark mode	S

4.3 Course registration (FR-REG)

ID	Requirement	Pri
FR-REG-01	The system shall list the courses offered for the student's programme, level and the current semester	M
FR-REG-02	The system shall refuse registration when registration is closed, a hold is set, no invoice exists, or less than 60 percent of the invoice is paid, and shall state the reason	M
FR-REG-03	The system shall enforce a credit load of 15 to 24 credits per semester	M
FR-REG-04	Saving a registration shall replace the student's earlier selection for that term in one transaction	M
FR-REG-05	The student shall be able to print a registration slip	S

4.4 Results (FR-RES)

ID	Requirement	Pri
FR-RES-01	The system shall show results by term with course, credits, score, grade and grade point	M
FR-RES-02	The system shall compute semester GPA, CGPA and the degree classification from the grade scale in section 7	M
FR-RES-03	The system shall identify failed courses (grade point below 1.5) for resit	M
FR-RES-04	The student shall be able to print a transcript	S

4.5 Fees and payments (FR-FEE)

ID	Requirement	Pri
FR-FEE-01	The system shall show each invoice with tuition, hall fee, SRC levy, total, amount paid, balance and status (Unpaid, Part-paid, Paid)	M
FR-FEE-02	A student shall be able to pay by MTN MoMo, Vodafone Cash, AirtelTigo Money, Visa or Mastercard, or Bank Deposit; the amount must be positive and not above the balance	M
FR-FEE-03	Each payment shall get a unique receipt number and a notification, and be printable as a receipt	M
FR-FEE-04	Payments in this release are recorded locally; no money is moved (see section 9)	M

4.6 Requests (FR-REQ)

ID	Requirement	Pri
FR-REQ-01	A student shall be able to submit resit, incomplete (IC), complaint, medical and internship requests, each stored with status Pending	M
FR-REQ-02	Complaints shall use the categories Academic, Finance, Hostel, ICT / Portal, Security and Other	M
FR-REQ-03	A student shall see the status and the staff response of each request they sent	M
FR-REQ-04	A resit shall carry a fee of GHS 50 per course	M
FR-REQ-05	Request statuses shall be Pending, In Review, Approved, Rejected, Resolved	M

4.7 Internship, evaluation, timetable and attendance

ID	Requirement	Pri
FR-INT-01	A student shall be able to add and delete internship log entries (date, activity, hours) and see total hours against the 240-hour target	M
FR-EVA-01	A student shall be able to rate and comment on each registered course once per term	S
FR-TT-01	The system shall show the student's weekly timetable (day, time, venue) for registered courses	M
FR-TT-02	The system shall show the student's attendance summary per course	S

4.8 Notifications and announcements (FR-NOT)

ID	Requirement	Pri
FR-NOT-01	The system shall notify a user on payment, registration saved, password change, request submitted and request reviewed	M
FR-NOT-02	The system shall show an unread count badge and let the user mark all as read	M
FR-NOT-03	Announcements shall be visible to all users, newest first	M

4.9 Administration and audit (FR-ADM, FR-AUD)

ID	Requirement	Role	Pri
FR-ADM-01	Dashboard with analytics (student, fee, request and registration counts)	Admin, Staff	M
FR-ADM-STU-01	Search and filter students by name, level and programme	Admin	M
FR-ADM-STU-02	Add a student: creates the login, the profile and the current-term invoice from level and hall fees	Admin	M
FR-ADM-STU-03	Edit a student, place or lift a registration hold, deactivate or reactivate, reset password, delete	Admin	M
FR-ADM-REQ-01	List requests by type and status; approve, reject or resolve with a response; the student is notified	Admin, Staff	M
FR-ATT-01	Mark attendance per course and date from the course roster	Admin, Staff	M
FR-ADM-ANN-01	Create and delete announcements	Admin, Staff	M
FR-ADM-REP-01	Export students, fees and results as CSV	Admin, Staff	M
FR-ADM-SET-01	Open or close registration for the current term	Admin	M

ID	Requirement	Role	Pri
FR-ADM-SET-02	Reset demo data to the seeded sample	Admin	S
FR-AUD-01	Record an audit entry for logins, failures, password changes, student changes, payments, registrations and request reviews, and show the latest entries	Admin	M

5. External interface requirements

- **User interface:** responsive layout; drawer menu on phones, sidebar on tablets and web; pull-to-refresh; validation messages and empty states on every screen; light and dark themes.
- **Hardware interface:** none beyond the device screen and touch input.
- **Software interfaces:** the SQLite database through expo-sqlite; expo-crypto for hashing and random codes; expo-print and expo-sharing for PDF and CSV output; the device browser for the university website link.
- **Communication interfaces:** none at run time in this release (fully offline). A production release needs HTTPS to a server API.

6. Non-functional requirements

Items marked (target) are proposed acceptance targets, not measured results.

ID	Category	Requirement
NFR-SEC-01	Security	Passwords salted and hashed; lockout after 5 failures; 6-digit reset codes stored hashed and expiring after 10 minutes
NFR-SEC-02	Security	A user can open only the screens allowed for their role, and a student sees only their own records
NFR-SEC-03	Security	Sensitive actions are written to the audit log
NFR-PER-01	Performance	(target) Any screen opens in under 2 seconds on a mid-range phone after first launch
NFR-USA-01	Usability	Same menu structure as the USTED portal; every form validates input and shows clear error text
NFR-REL-01	Reliability	Multi-step writes (registration, student creation) run in SQLite transactions; a schema version is tracked for migrations
NFR-POR-01	Portability	One code base for Android, iOS and web
NFR-MNT-01	Maintainability	All SQL in src/db/repo.js; all rules in src/config.js; screens, components and navigation kept separate
NFR-AVL-01	Availability	Works fully offline after first launch

7. Business rules

ID	Rule	Source in code
BR-01	At least 60 percent of the invoice must be paid before registering	RULES.minPaymentPercent
BR-02	Credit load is 15 to 24 per semester	RULES.minCredits, maxCredits
BR-03	5 failed logins lock the account for 10 minutes	RULES.maxLoginAttempts, lockMinutes
BR-04	Reset codes expire after 10 minutes and work once	RULES.resetCodeMinutes
BR-05	Passwords need 8 or more characters, a letter and a number	RULES.minPasswordLength
BR-06	Resit fee is GHS 50 per course	RULES.resitFeePerCourse
BR-07	Internship target is 240 hours	RULES.internshipRequiredHours
BR-08	Invoice = academic fee (Level 100: 2,200; 200: 2,100; 300: 2,000; 400: 2,500) + hall fee (0 to 900) + SRC levy 100, all in GHS	ACADEMIC_FEES, HALL_FEES, APP.srcLevy
BR-09	Grades: 80+ A (4.0), 75 A- (3.75), 70 B+ (3.5), 65 B (3.25), 60 B- (3.0), 55 C+ (2.75), 50 C (2.5), 45 C- (2.25), 40 D+ (2.0), 35 D (1.75), 30 D- (1.5), below 30 F (0.0)	GRADE_SCALE
BR-10	A grade point below 1.5 is a fail and the course needs a resit	PASS_POINT

ID	Rule	Source in code
BR-11	Classification by CGPA: First Class 3.6 and above; Second Upper 3.0; Second Lower 2.5; Third Class 2.0; Pass 1.5; Fail below 1.5	CLASSIFICATION
BR-12	Registration also needs registration open for the term, no hold, and an invoice for the term	registrationEligibility()

8. Data requirements

The system keeps 18 tables: users, students, terms, courses, registrations, results, invoices, payments, requests, internship_logs, evaluations, announcements, notifications, timetable, attendance, password_resets, audit_log and kv (key-value store holding the session). Foreign keys with cascade deletes enforce referential integrity. The entity-relationship diagram is in Part B and the table design in Part C.

9. Constraints, risks and known limitations

The README already flags these. They are accepted limitations of v2.0.0 and the main inputs to the next release.

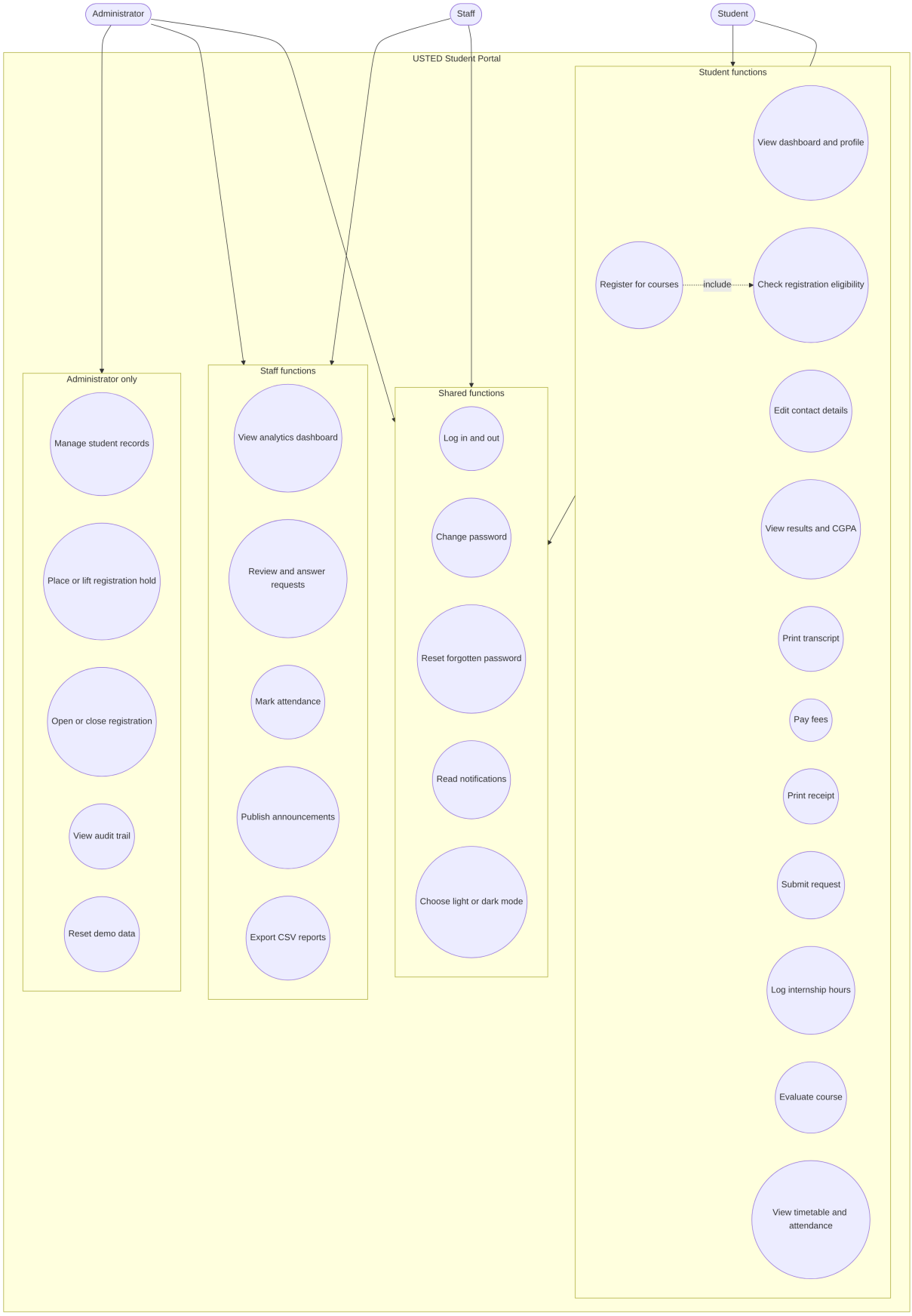
#	Limitation	Impact	Recommended action
L1	Database and login checks run on each device	A client-side check is not a security boundary, and data is not shared between devices	Move authentication and data to a server API with bcrypt or argon2 and server-side permission checks
L2	Payments are only recorded locally	No real money moves	Integrate a MoMo or card gateway on the server
L3	Reset codes are shown on screen in demo mode	Anyone holding the device can read the code	Send codes through an SMS, e-mail or Telegram provider
L4	Seeded demo passwords (student123, staff123, admin123) and a demo hint on the login screen	Known credentials	Change seeded passwords and remove the hint before real use
L5	New students and admin password resets get the default password student123	Predictable first password	Force a password change at first login

Part B: UML diagrams

All diagrams are Mermaid text, so they render in the document and can be edited as code. They describe the behaviour implemented in v2.0.0.

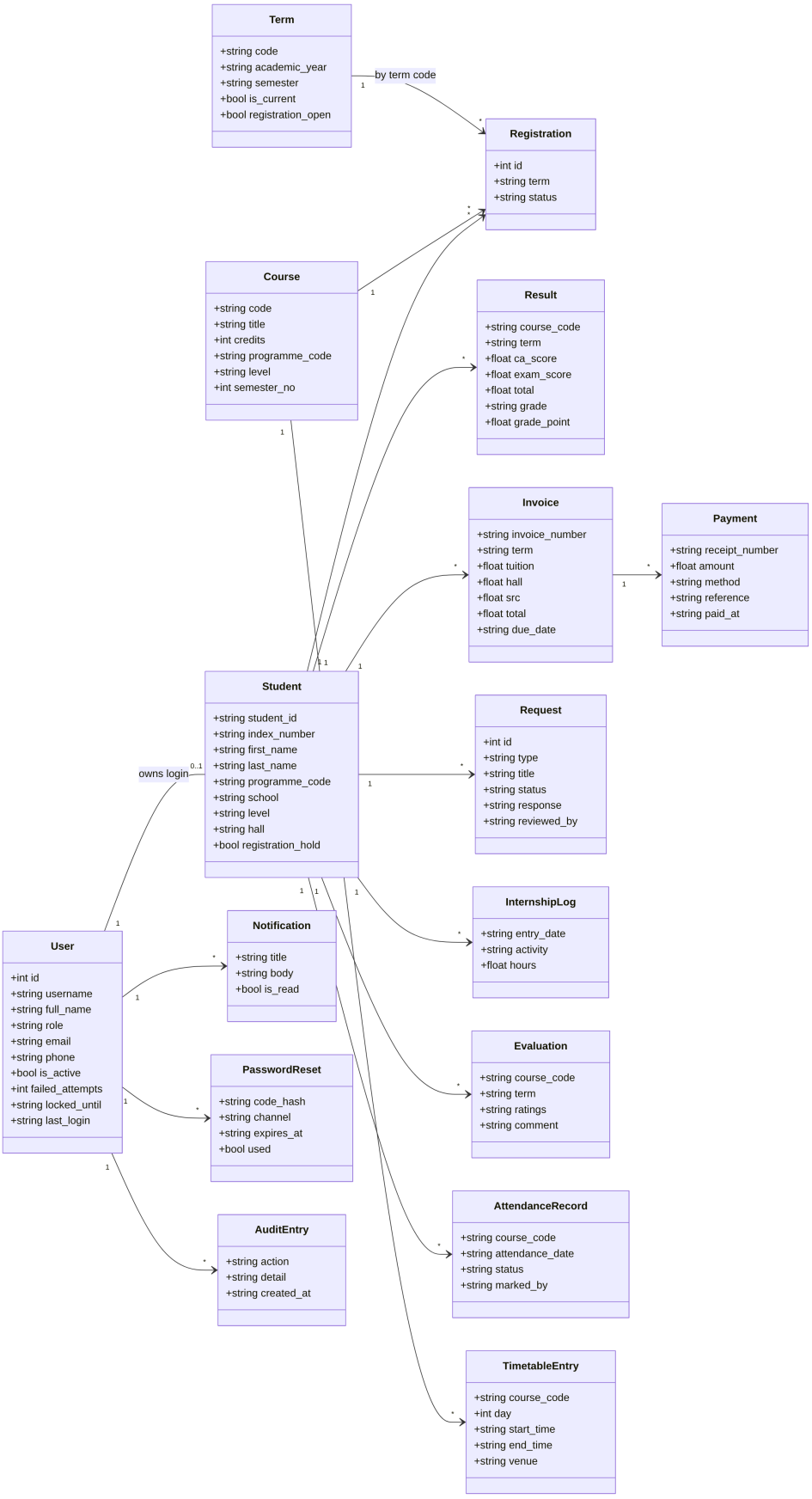
B1. Use case diagram

Students get the shared and student functions. Staff get the shared and staff functions. The administrator gets all three groups plus the administrator-only functions.



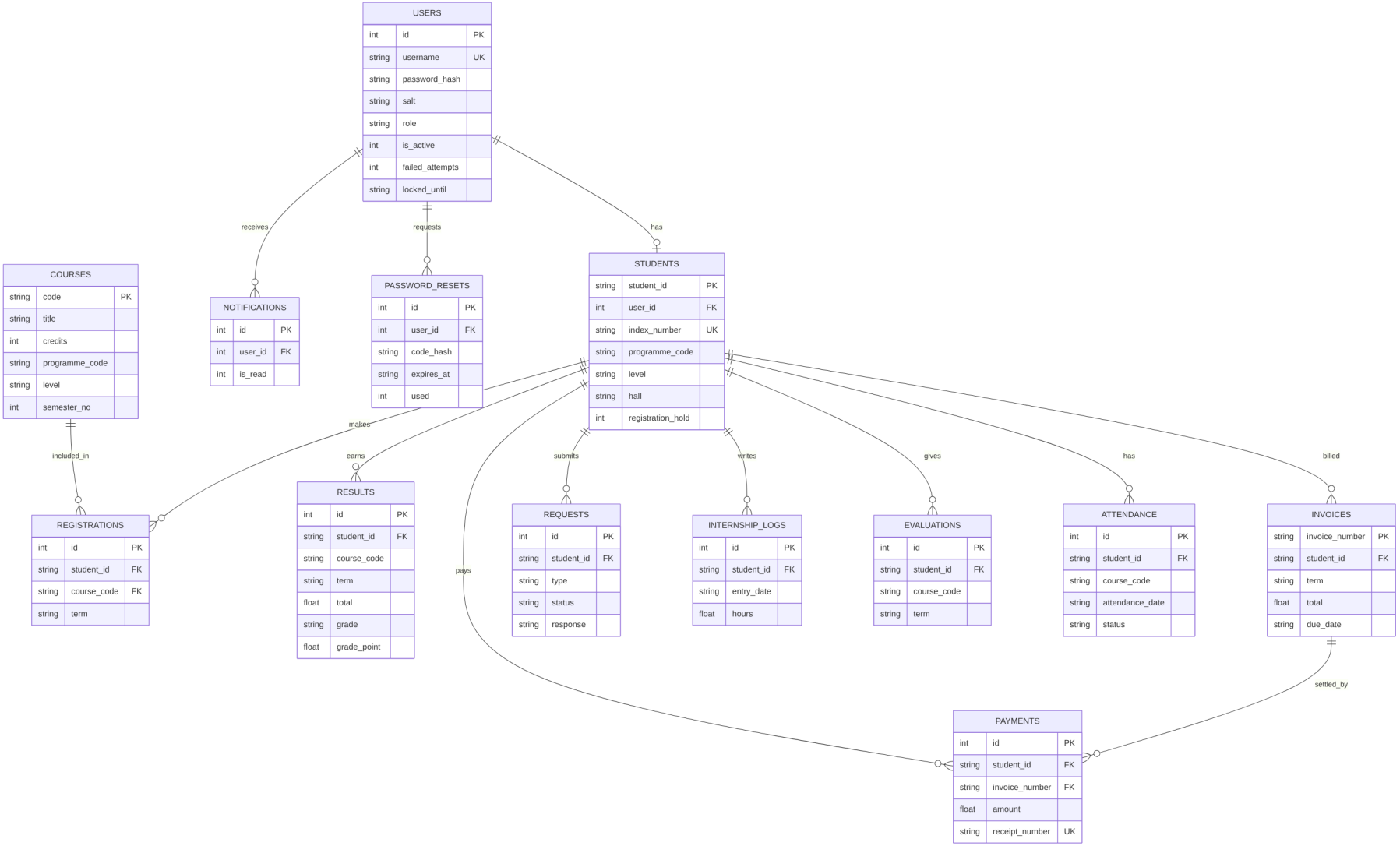
B2. Class diagram (domain model)

The classes match the SQLite tables. Cascade deletes run from User and Student to everything that belongs to them.

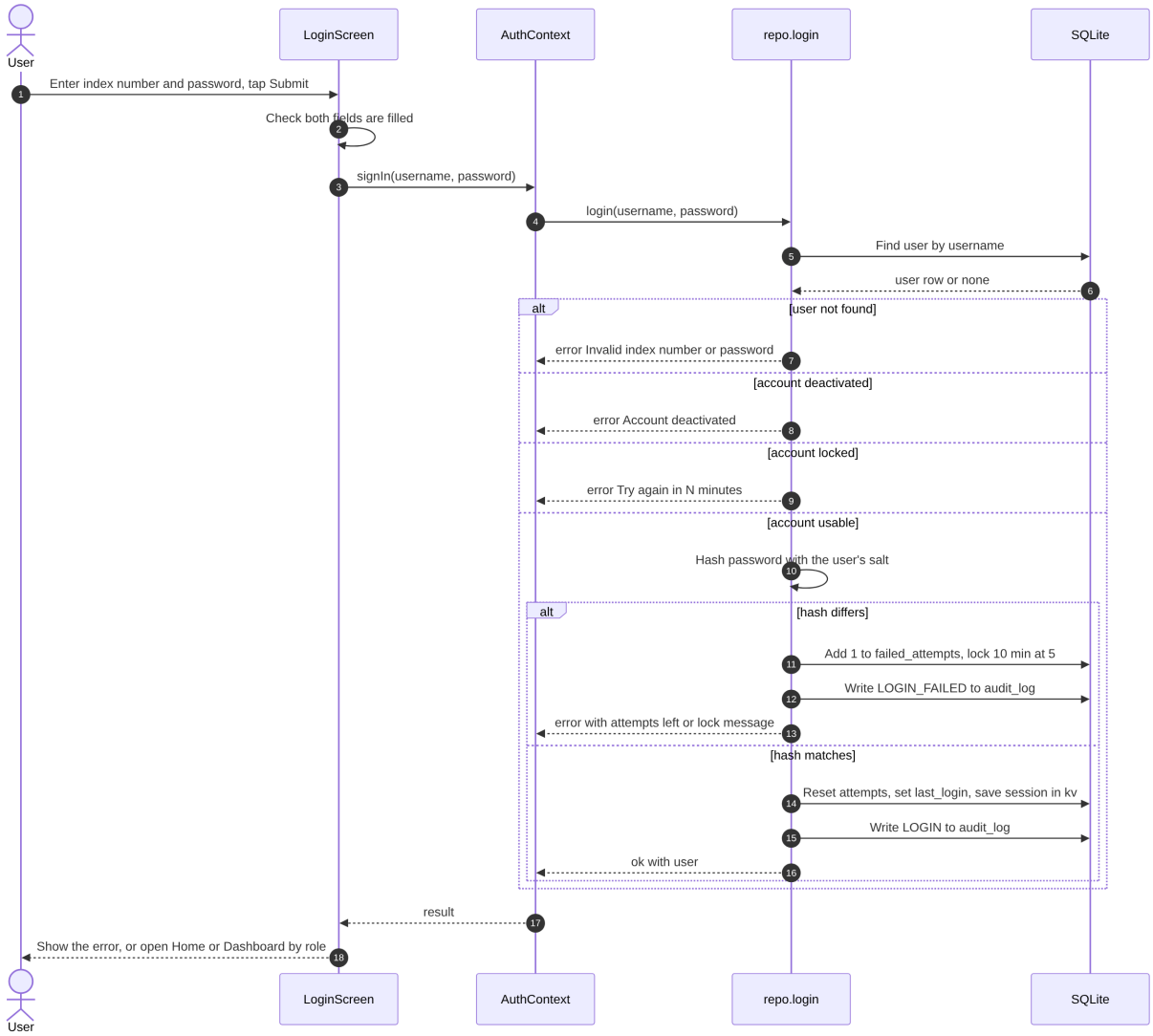


B3. Entity-relationship diagram (database)

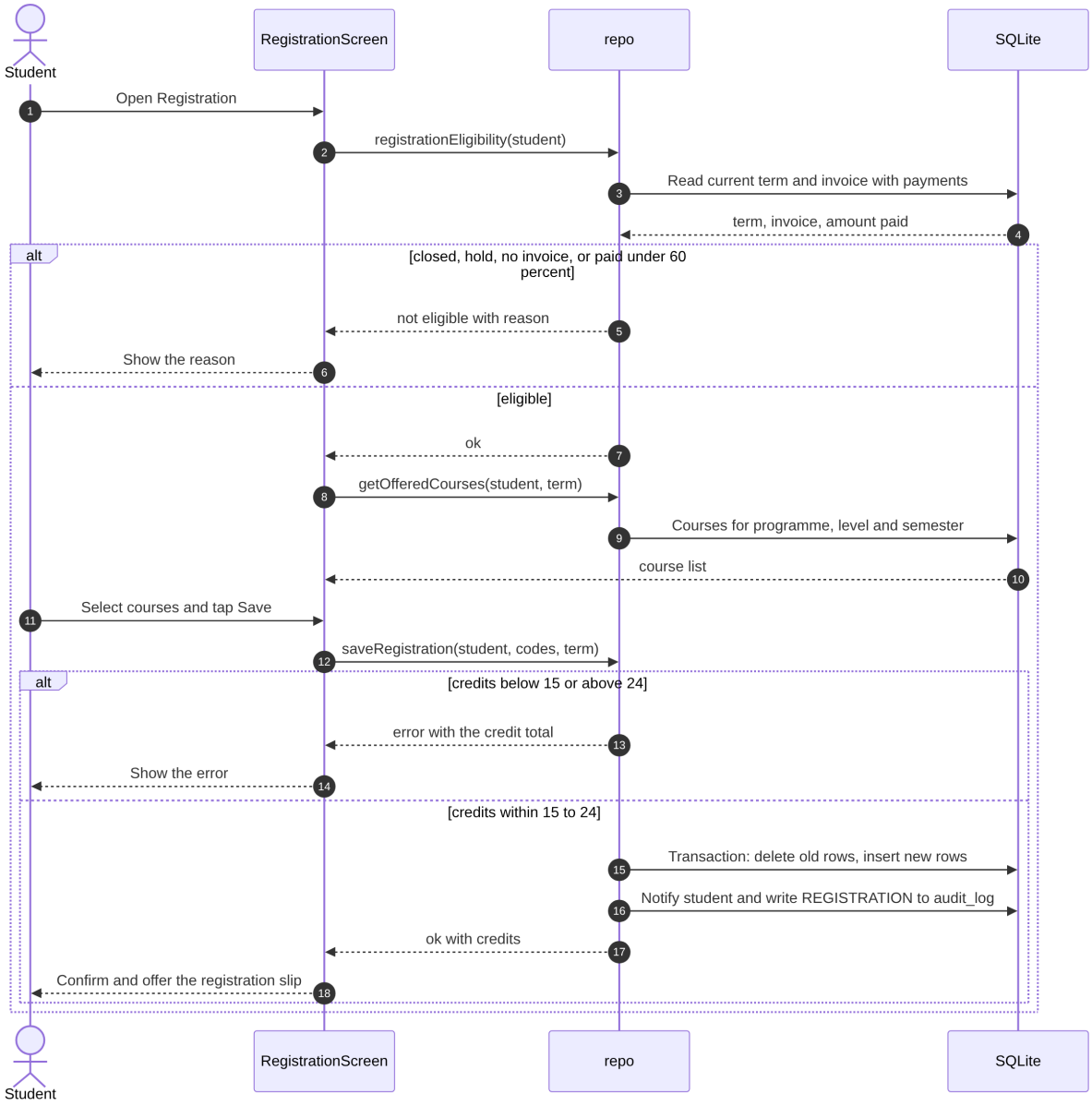
Key columns only; the full column list is in section C4. Results, timetable and attendance store the course code as text without a foreign key to courses.



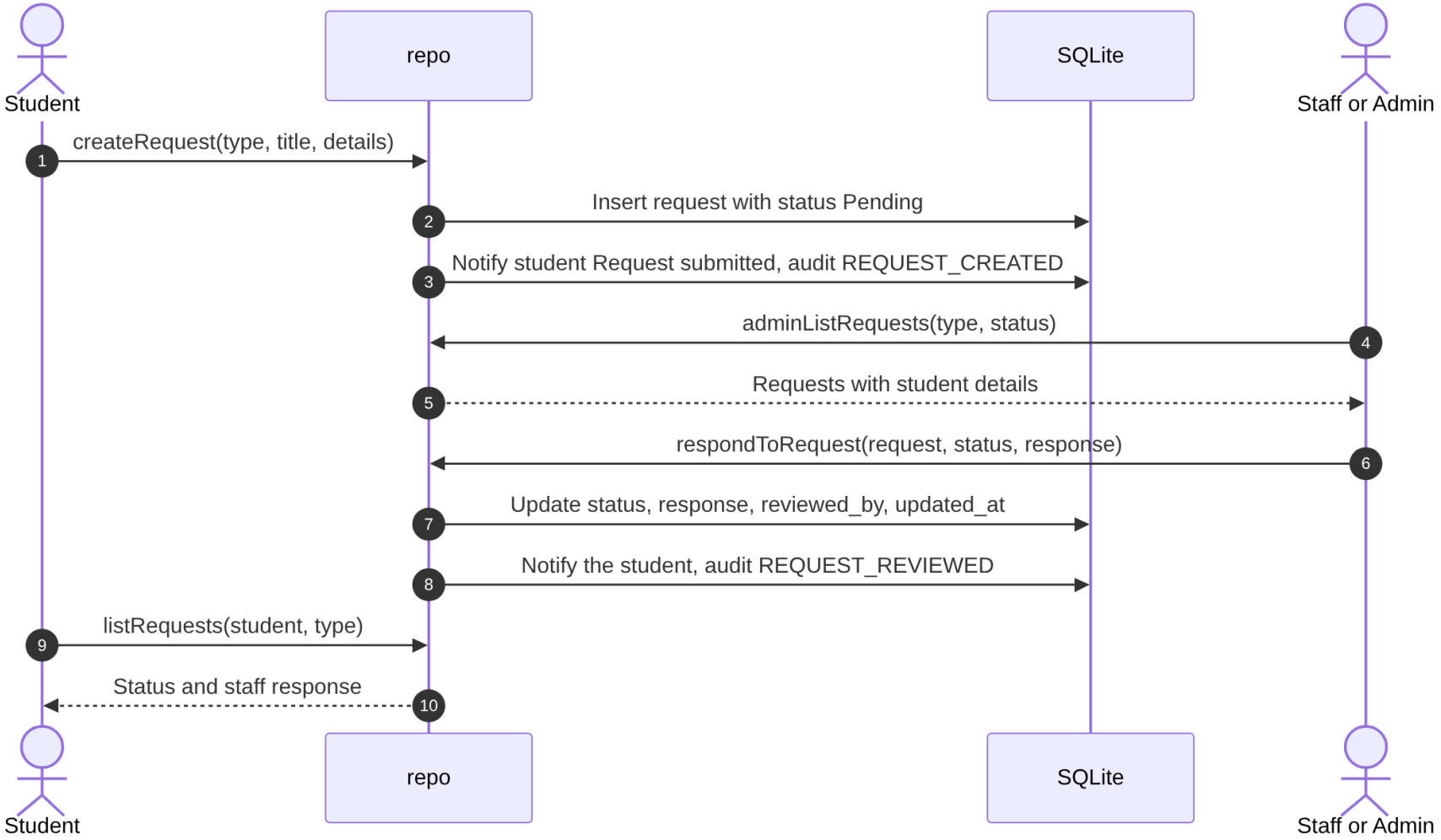
B4. Sequence diagram: login with lockout



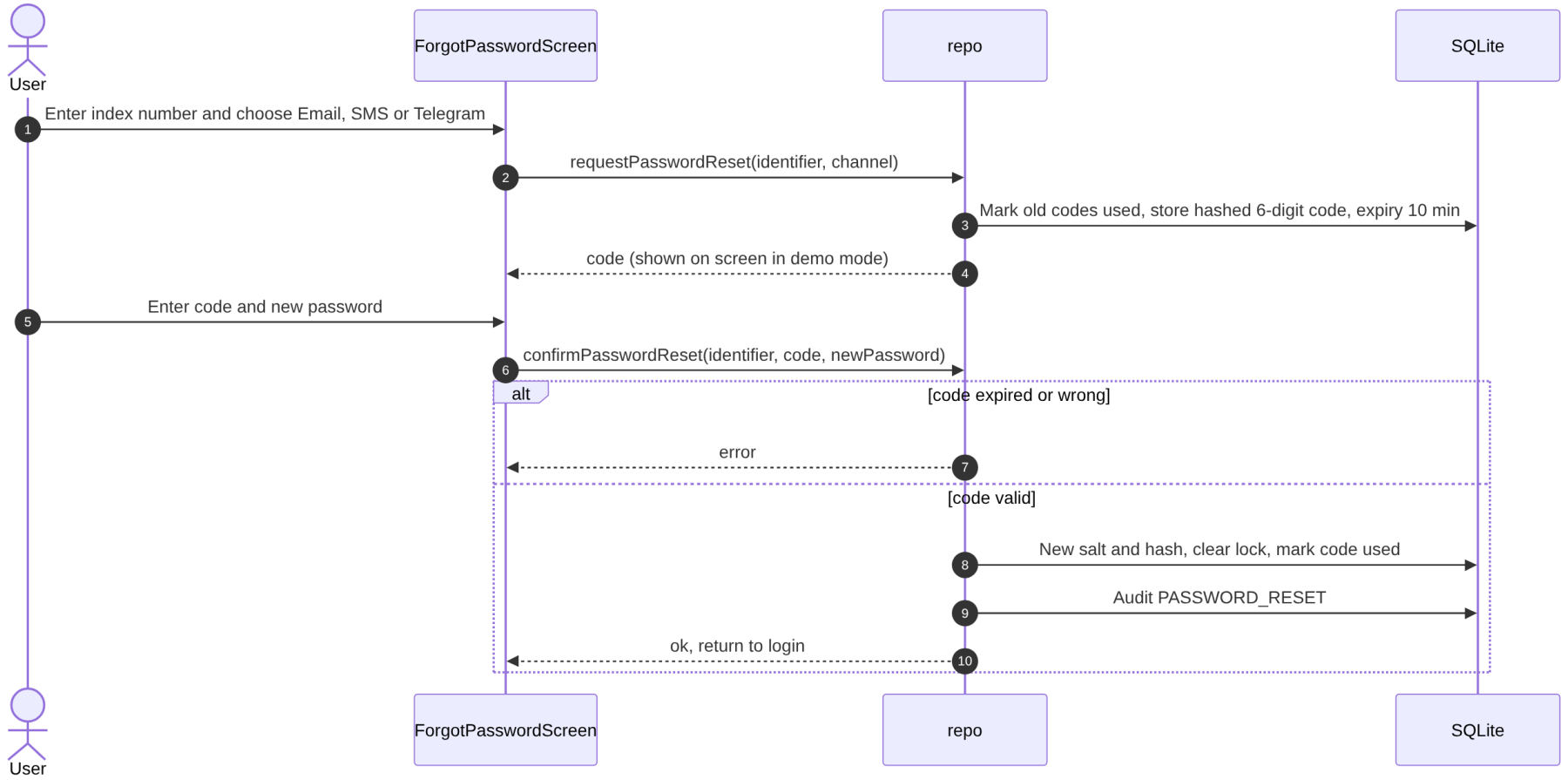
B5. Sequence diagram: course registration



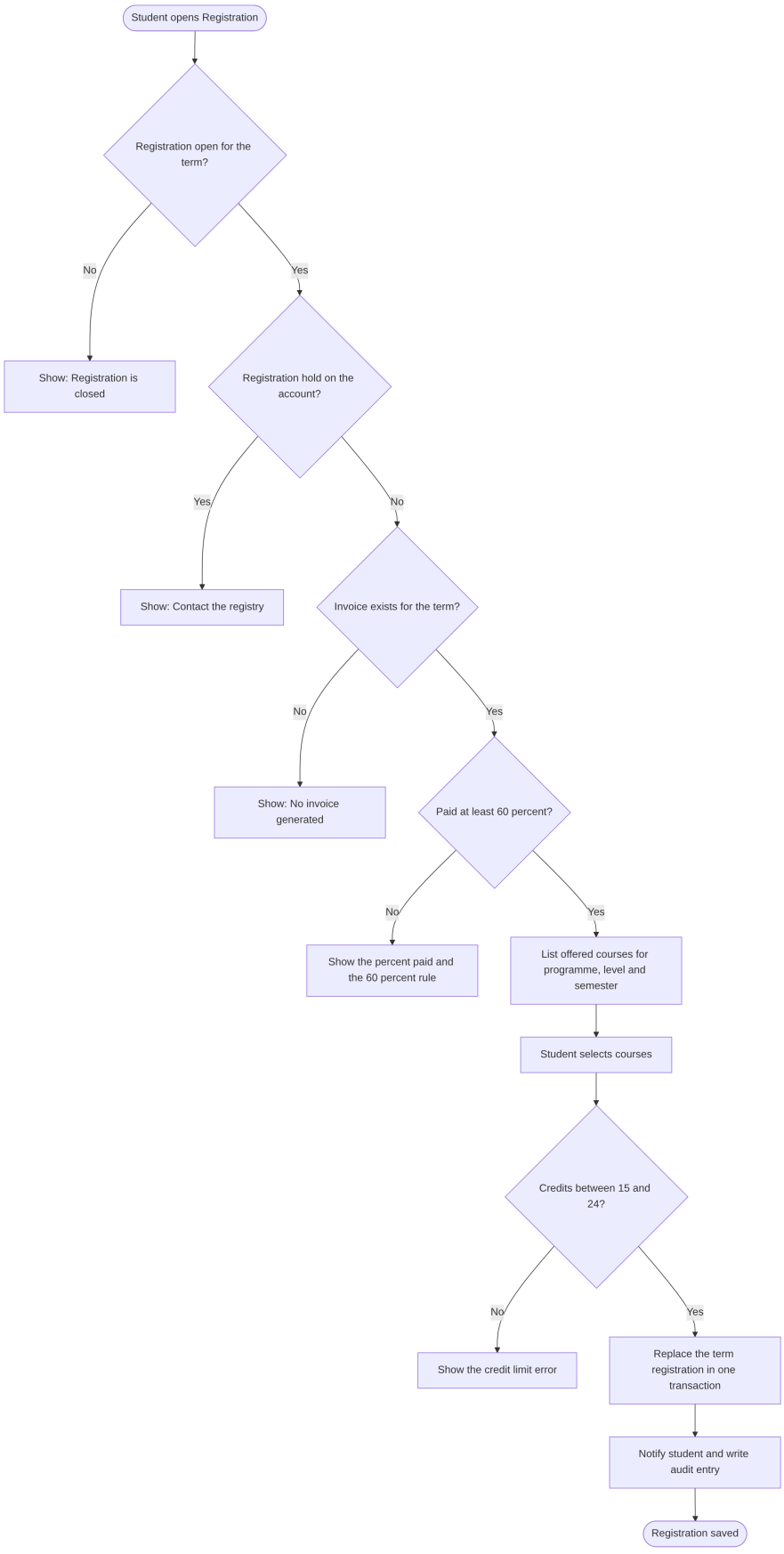
B6. Sequence diagram: request submitted and reviewed



B7. Sequence diagram: forgotten password

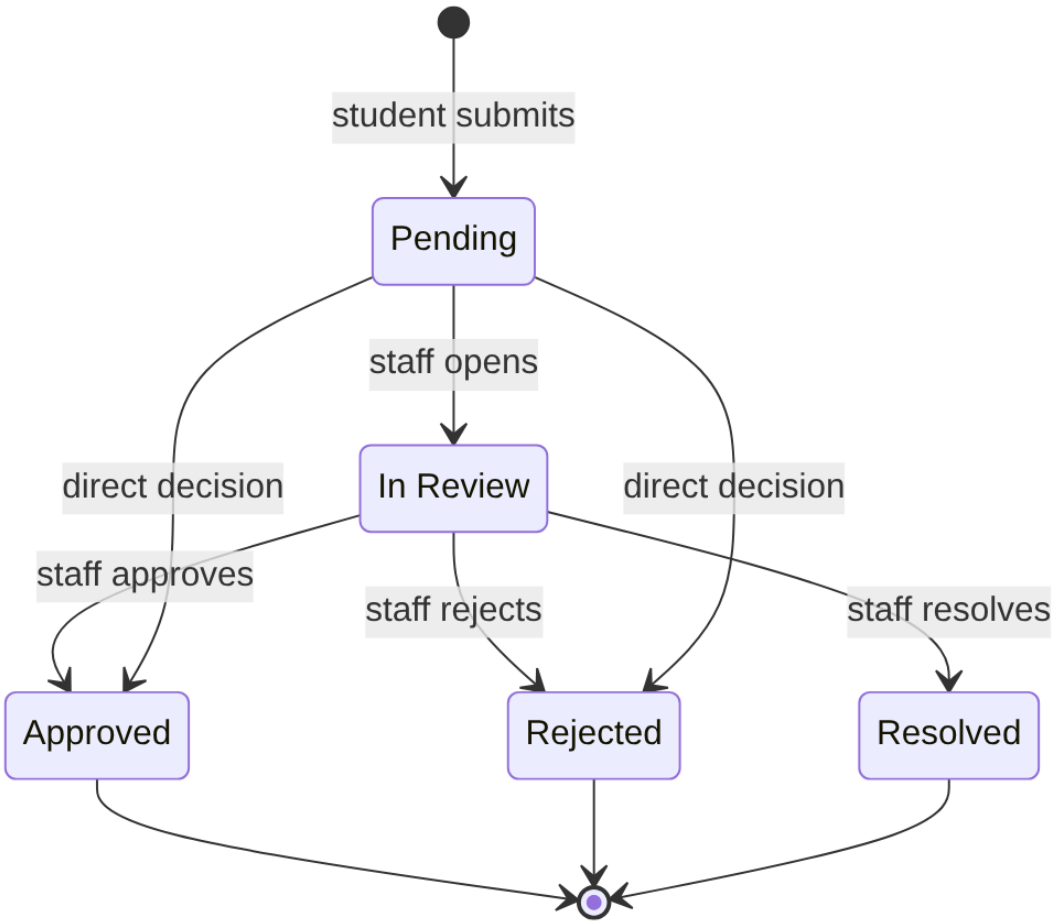


B8. Activity diagram: registration eligibility and save

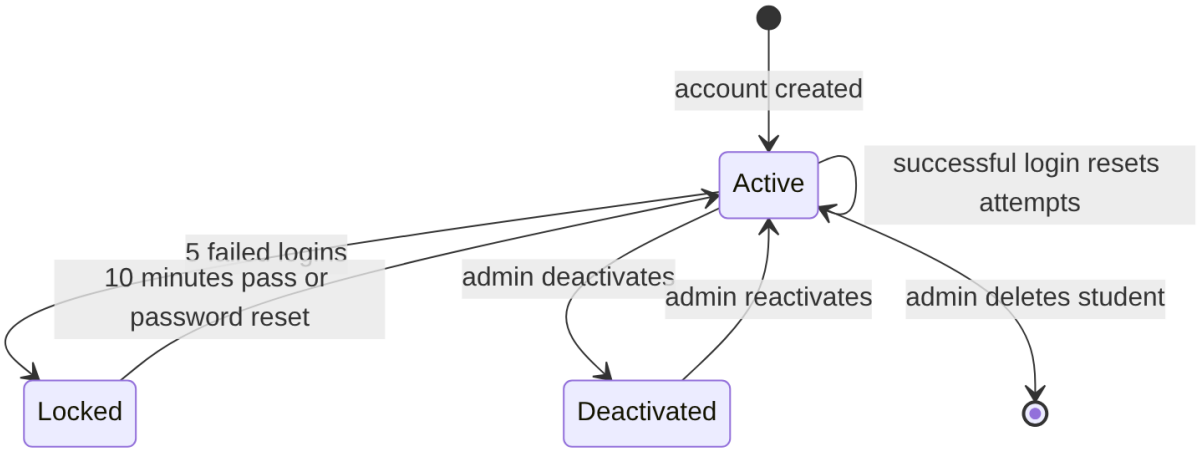


B9. State diagrams

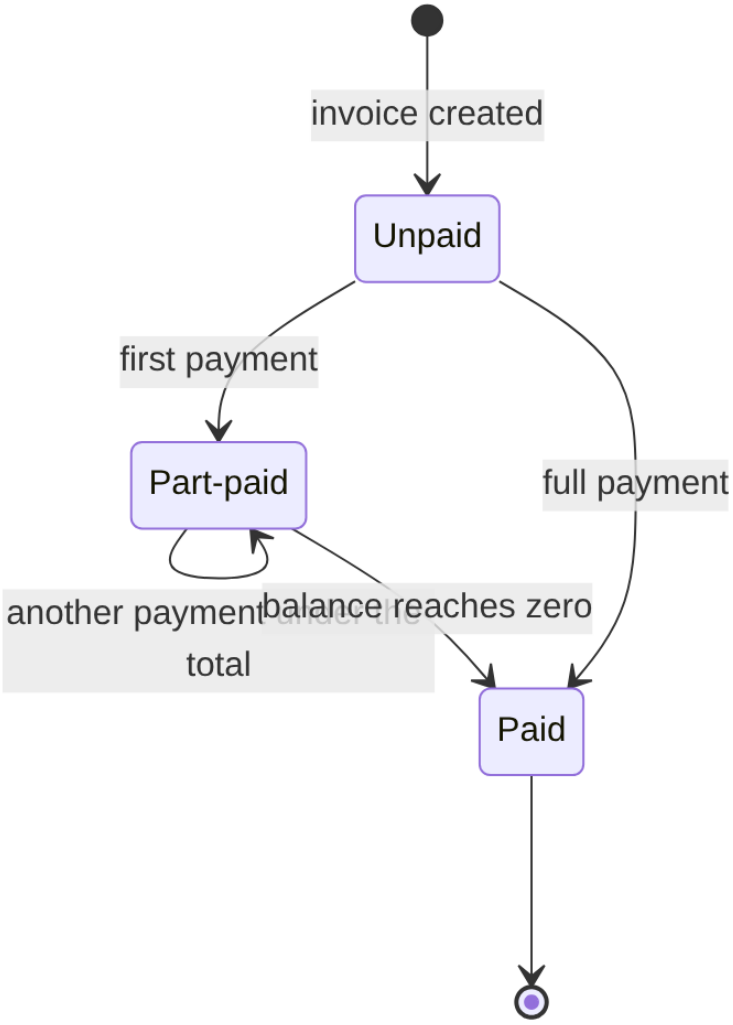
Request lifecycle. Staff can set any of the five statuses; the usual path is shown.



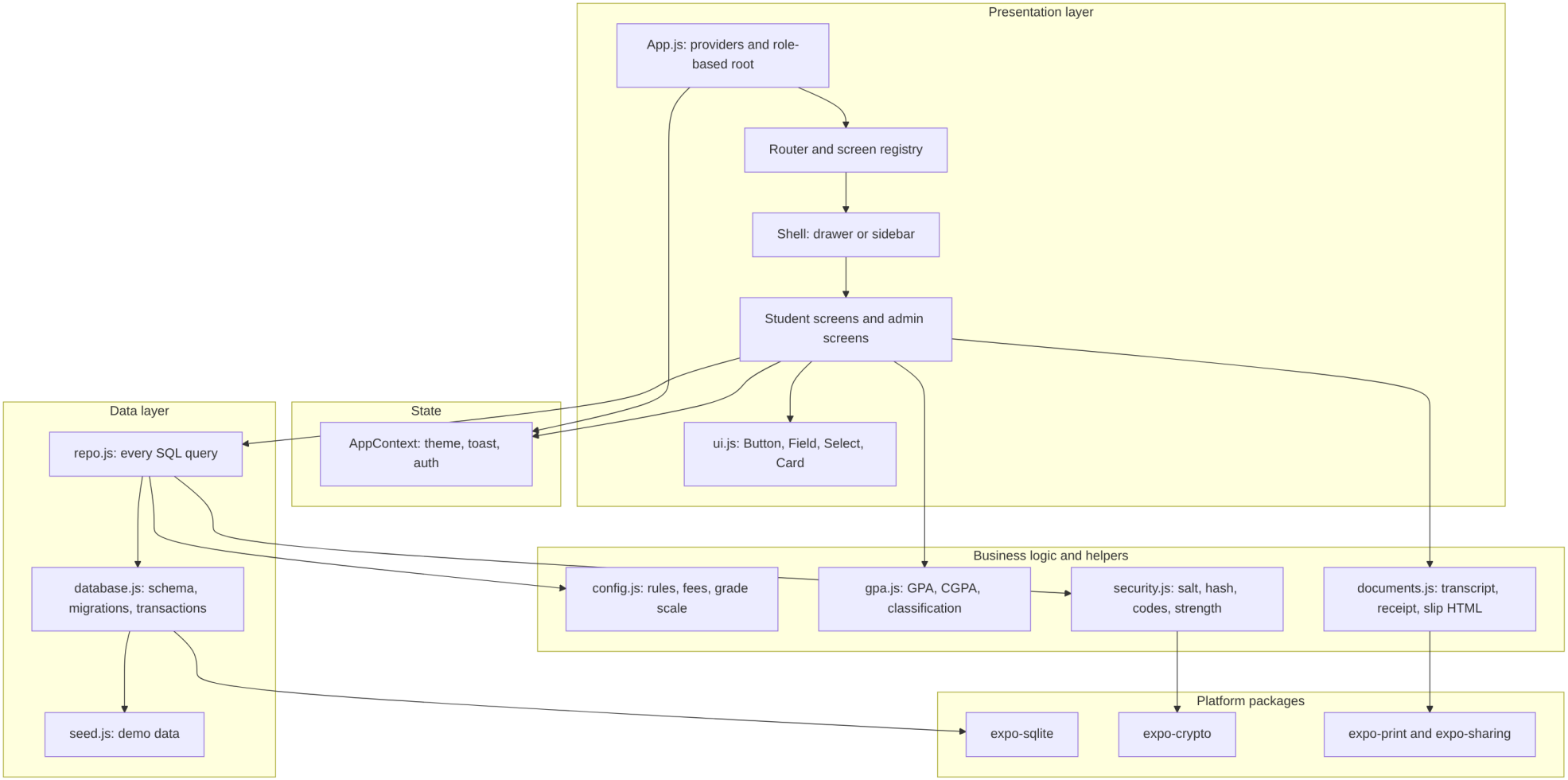
User account lifecycle.



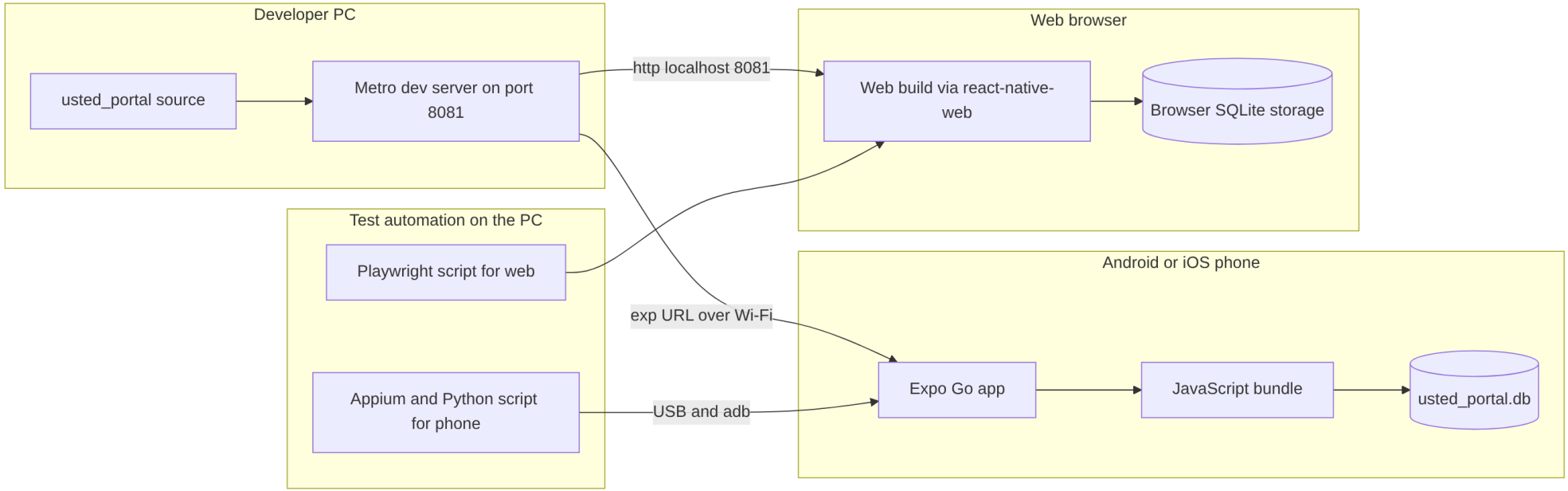
Invoice status.



B10. Component diagram



B11. Deployment diagram



Part C: Design documentation

C1. Design goals

1. One code base for Android, iOS and web.
2. Work fully offline, with data in on-device SQLite.
3. Keep all SQL in one file and all institution rules in another, so a server API or new rules can be added without touching the screens.
4. Mirror the USTED portal look and menu, and keep admin tools from the original ERP.

C2. Architecture

The app is layered, as shown in diagram B10.

Layer	Files	Responsibility
Presentation	App.js, src/screens, src/components, src/navigation	Screens, forms, drawer or sidebar, routing by role
State	src/context/AppContext.js	Theme (light or dark), toast messages, signed-in user
Business logic and helpers	src/config.js, src/utils	Rules, fees, grade scale, GPA, hashing, validation, PDF HTML
Data	src/db/database.js, repo.js, seed.js	Schema, migrations, transactions, every query, demo data
Platform	expo-sqlite, expo-crypto, expo-print, expo-sharing	Storage, hashing and random codes, PDF and CSV output

Design patterns used:

- **Repository:** repo.js is the only file that runs SQL, so screens call functions such as login(), saveRegistration() and makePayment().
- **Provider and context:** ThemeProvider, ToastProvider and AuthProvider wrap the app (App.js).
- **Screen registry and role guard:** src/navigation/screens.js maps each screen name to a component and its allowed roles; the Screens component in App.js falls back to the role's home screen when a role may not open a screen.
- **Role-specific menus:** STUDENT_MENU, ADMIN_MENU and STAFF_MENU in src/navigation/menu.js.
- **Responsive shell:** a permanent sidebar at 900 px or wider, an animated drawer below that.
- **Startup gate:** the Gate component opens and seeds the database, shows a splash with a Try again button if it fails, then shows login or the portal.

C3. Module design

Module	Main functions or content
App.js	Providers; Gate (database start); Root (login, forgot password or portal); Screens (role guard)
src/db/database.js	getDb, all, one, run, exec, transaction, initDatabase, resetDatabase, audit, notify, key-value helpers
src/db/repo.js	Auth: login, logout, restoreSession, changePassword, requestPasswordReset, confirmPasswordReset. Students: listStudents, createStudent, updateStudent, updateContact, setStudentFlag, resetStudentPassword, deleteStudent. Academics: getResults, failedCourses, registrationEligibility, getOfferedCourses, saveRegistration, studentTimetable, attendanceSummary. Finance: getInvoices, getPayments, makePayment. Requests: createRequest, listRequests, adminListRequests, respondToRequest. Other: internship logs, evaluations, announcements, notifications, saveAttendance, dashboardStats, reports, auditTrail, setRegistrationOpen
src/db/seed.js	Seeds the terms, courses, 10 demo students, staff, admin, invoices, payments, results and sample data on first launch

Module	Main functions or content
src/utils/gpa.js	gradeFromScore, calcGPA, classify, isFail, summarise
src/utils/security.js	newSalt, hashPassword, randomCode, passwordIssues, passwordStrength
src/utils/documents.js	transcriptHTML, receiptHTML and the registration slip as printable HTML
src/utils/format.js, validate.js, hooks.js, share.js	Money and date formatting, validation, data-loading hook, file sharing
src/components/ui.js	Button, Field (with show and hide password), Select, IconButton, Card, Avatar and other shared controls
src/components/Shell.js	Header, drawer, sidebar, notification bell with unread badge

C4. Database design

SQLite file usted_portal.db. Foreign keys are on. A schema version is kept with PRAGMA user_version (currently 1), and the database is seeded when the users table is empty.

Table	Key columns	Notes
users	id PK, username UNIQUE, password_hash, salt, role (admin, staff or student), is_active, failed_attempts, locked_until, last_login	One row per login
students	student_id PK, user_id UNIQUE FK to users, index_number UNIQUE, names, programme, programme_code, school, level, hall, registration_hold	Deleting a user cascades here
terms	code PK, academic_year, semester, is_current, registration_open	Current term drives registration
courses	code PK, title, credits, programme_code, level, semester_no, type, lecturer	Offered by programme, level and semester
registrations	id PK, student_id FK, course_code FK, term, status; UNIQUE (student_id, course_code, term)	No duplicate registration
results	id PK, student_id FK, course_code, course_title, credits, term, ca_score, exam_score, total, grade, grade_point	Index on (student_id, term)
invoices	invoice_number PK, student_id FK, term, tuition, hall, src, total, due_date	One per term
payments	id PK, student_id FK, invoice_number FK, amount, method, reference, receipt_number UNIQUE, paid_at	Index on invoice_number
requests	id PK, student_id FK, type, title, details, extra, status, response, reviewed_by, created_at, updated_at	Index on (student_id, type)
internship_logs	id PK, student_id FK, entry_date, activity, hours	Sum of hours versus 240
evaluations	id PK, student_id FK, course_code, term, ratings, comment; UNIQUE (student_id, course_code, term)	One evaluation per course per term
announcements	id PK, title, body, author, created_at	Shown to everyone
notifications	id PK, user_id FK, title, body, is_read	Index on (user_id, is_read)
timetable	id PK, course_code, day, start_time, end_time, venue	Joined to registrations
attendance	id PK, student_id FK, course_code, attendance_date, status, marked_by; UNIQUE (student_id, course_code, attendance_date)	One mark per day
password_resets	id PK, user_id FK, code_hash, salt, channel, expires_at, used	Code stored hashed
audit_log	id PK, user_id, action, detail, created_at	Append-only trail
kv	key PK, value	Holds the signed-in user id (session_user_id)

C5. Screens and access control

Screen	Student	Staff	Admin
Home, My Profile, Registration, Results, Fees, Resits and ICs, Internship, Complaint, Medicals, Evaluation, Timetable	Yes	No	No
Notifications, Settings	Yes	Yes	Yes
Dashboard, Requests, Attendance, Announcements, Reports and Export	No	Yes	Yes
Students (manage records)	No	No	Yes

A user who tries to open a screen outside their role is sent to their home screen (Home for students, Dashboard for staff and admin).

C6. Security design

Control	How it works
Password storage	SHA-256 of salt:password with a random per-user salt (expo-crypto); the hash is never sent to the UI because the signed-in user object leaves it out
Brute force	5 failed attempts lock the account for 10 minutes; the counter resets on success or password reset
Password policy	At least 8 characters, a letter and a number; a 0 to 4 strength score
Reset codes	6 digits from secure random bytes, stored as salted hash, valid 10 minutes, single use, older codes invalidated
SQL injection	Every query uses ? placeholders with a parameter array
Authorisation	Role guard in Screens and role lists in the screen registry
Accountability	audit_log entries for login, failed login, password change and reset, student changes, payments, registrations and request reviews
Known gap	All checks run on the device, so they are not a security boundary (limitation L1 in section 9)

C7. Key algorithms and rules

Grade from score: round the score, then take the first row of the grade scale (80 and above first) whose minimum is not above the score.

GPA and CGPA, rounded to 2 decimals (c = credits, g = grade point):

$$\text{GPA} = \Sigma(c \times g) / \Sigma c$$

CGPA uses the same formula over all terms. The degree class comes from the CGPA table in BR-11.

Invoice balance = total minus the sum of payments. Status is Paid when paid is at least the total, Part-paid when paid is above zero, otherwise Unpaid. The paid percentage used for registration is round(paid divided by total times 100).

Lockout: attempts = failed_attempts + 1. At 5, locked_until is set 10 minutes ahead and the counter is reset to 0.

C8. User interface design

- **Look:** maroon, teal and blue portal palette (theme.js), with light and dark themes; printouts use maroon headers.
- **Navigation:** login, then a drawer (phones) or sidebar (900 px and wider) with Home, Registration, Results, Fees, Resits and ICs, Internship, Complaint, Medicals, Evaluation, and a More group with Timetable, My Profile and Settings.
- **Login screen:** logo, Index Number and Password fields (password can be shown or hidden), Submit, three forgot-password links, and a demo-account hint.
- **Feedback:** toast messages, inline field errors, empty states, pull-to-refresh, busy indicators on buttons.
- **Documents:** transcript, receipt and registration slip are generated as HTML and printed or shared as PDF; reports are shared as CSV.

C9. Validation and error handling

Situation	Behaviour
Empty login fields	Message: Enter your index number and password
Wrong login	Generic message with attempts left; a lock message after the fifth failure
Database cannot open	Splash screen with the error and a Try again button
Registration not allowed	The exact reason (closed, hold, no invoice, under 60 percent)
Credit load out of range	Message with the selected credit total
Payment above balance or not positive	Message and no record written
Duplicate index number on add student	Message that the student already exists
Expired or wrong reset code	Message to request a new code, or Incorrect code

C10. Test design

Strategy: manual and automated functional tests on the demo data, run on the web build and on a real phone. Automated login and menu tours are provided as two Python scripts: auto_login.py (Playwright, web) and auto_login_phone.py (Appium, Android with Expo Go). They log in with the demo accounts and open every menu item for the role.

ID	Scenario	Expected result
TC-01	Log in as 5211041001 with student123	Student Home opens
TC-02	Log in with a wrong password	Error with 4 attempts left
TC-03	Wrong password 5 times	Account locked for 10 minutes
TC-04	Log in as staff and open Students	Not offered in the menu; direct access falls back to the Dashboard
TC-05	Register with fewer than 15 credits	Minimum credit load error
TC-06	Register with more than 24 credits	Maximum credit load error
TC-07	Admin closes registration, student opens Registration	Registration is closed message
TC-08	Pay more than the balance	Amount is more than the outstanding balance
TC-09	Pay a valid amount	Receipt number issued and notification created
TC-10	Submit a complaint	Status Pending and notification Request submitted
TC-11	Staff approves the request with a response	Student receives a notification with the status
TC-12	Reset password with a code older than 10 minutes	Code expired message
TC-13	Admin adds a student	Login created with default password and a current-term invoice
TC-14	Set a new password of 6 letters only	Rejected: needs 8 characters and a number
TC-15	Mark attendance and save	Records stored once per student, course and date
TC-16	Export students as CSV	File shared with all student rows

C11. Build, run and deployment

1. Install dependencies: npm install, then npx expo install for the packages listed in the README.
2. Start: npx expo start. Scan the QR code with Expo Go, or press w for the browser. After changing the SDK or after errors, run npx expo start -c to clear the cache.
3. The database is created and seeded on first launch. Admin, Settings, Reset demo data restores the sample data.
4. For production, build with EAS or publish the web build, and add the server described in section 9.

C12. Requirements traceability

Requirements	Screen	Main functions (repo.js unless stated)
FR-AUTH-01 to 05	LoginScreen, App.js	login, restoreSession, logout
FR-AUTH-06, 07	ProfileScreen (Security tab)	changePassword; passwordIssues and passwordStrength in security.js
FR-AUTH-08	ForgotPasswordScreen	requestPasswordReset, confirmPasswordReset
FR-AUTH-09	App.js, screens.js, menu.js	role guard in Screens
FR-PRO	HomeScreen, ProfileScreen, SettingsScreen	getStudentByUser, updateContact, dashboard data
FR-REG	RegistrationScreen	registrationEligibility, getOfferedCourses, saveRegistration
FR-RES	ResultsScreen	getResults, failedCourses; summarise in gpa.js; transcriptHTML
FR-FEE	FeesScreen	getInvoices, getPayments, makePayment; receiptHTML
FR-REQ	ResitsScreen, ComplaintScreen, MedicalsScreen, InternshipScreen	createRequest, listRequests
FR-INT, FR-EVA, FR-TT	InternshipScreen, EvaluationScreen, TimetableScreen	addLog, deleteLog, saveEvaluation, studentTimetable, attendanceSummary
FR-NOT	NotificationsScreen, HomeScreen, Shell	listNotifications, unreadCount, markAllRead, listAnnouncements
FR-ADM-STU	AdminStudents	listStudents, createStudent, updateStudent, setStudentFlag, resetStudentPassword, deleteStudent
FR-ADM-REQ	AdminRequests	adminListRequests, respondToRequest
FR-ATT	AdminAttendance	courseRoster, attendanceFor, saveAttendance
FR-ADM-ANN	AdminAnnouncements	createAnnouncement, deleteAnnouncement
FR-ADM-REP, FR-ADM-01	AdminReports, AdminHome	dashboardStats, reportStudents, reportFees, reportResults
FR-ADM-SET, FR-AUD	AdminReports, SettingsScreen	setRegistrationOpen, resetDatabase, auditTrail

C13. Recommended next steps

1. Add a server API and central database (the first item in the README), and switch repo.js functions to API calls.
2. Move password hashing to bcrypt or argon2 on the server and enforce roles server-side.
3. Integrate a real MoMo or card gateway and a real SMS, e-mail or Telegram provider for reset codes.
4. Force a password change at first login and remove the demo hint and demo passwords.
5. Add unit tests for gpa.js, security.js and the repo.js business rules, and keep the two automation scripts as end-to-end smoke tests.